

Operating Systems Project Phase 4 Report

Farah Elnakhal - fqe9080

Myra Rafiq - mr7316

5th May 2026

1 Architecture and Design

The system is structured around three main components: the main server thread, client handler threads, and a scheduler thread. The main thread is responsible for accepting incoming client connections, while each client is handled in its own thread to allow multiple clients to interact with the server concurrently.

The scheduler thread simulates a single CPU. Even though multiple clients may submit requests at the same time, only one task is executed at any given moment. All program requests (i.e. `./demo N`) are inserted into a shared waiting queue, and the scheduler selects which task to run based on the scheduling algorithm.

A key design decision was to clearly separate execution paths:

- Shell commands are executed immediately using `execute_and_capture()`, since they do not require scheduling.
- Program commands (`./demo N`) are converted into `task_t` objects and passed to the scheduler queue.

This separation keeps the system efficient and avoids unnecessary scheduling overhead for simple commands.

To support safe concurrency, a mutex is used to protect access to the shared queue, and a semaphore is used to signal when new tasks are available. This ensures that the scheduler thread sleeps when there is no work and wakes up only when needed.

Each task stores all required metadata (client ID, burst time, remaining time, socket, etc.), allowing the scheduler to pause and resume execution correctly without restarting the process.

2 Implementation Highlights

The main functionality implemented on the server side was integrating the scheduler with the existing multithreaded server from Phase 3.

When a client sends a command, the server first determines its type. If the command matches the format `./demo N`, a new `task_t` structure is created and passed to the scheduler using `scheduler_enqueue()`. Otherwise, the command is executed immediately and its output is sent back to the client.

The scheduler thread is initialized once at server startup using `scheduler_init()` and runs continuously in parallel. This allows the server to keep accepting new clients while scheduling tasks in the background.

When a client exits, the server calls `scheduler_remove_client(client_id)` to ensure that all tasks associated with that client are removed from the queue or terminated if currently running. This prevents orphaned processes and keeps the system stable.

Server-side logging was also implemented to clearly show system behavior. Each client action logs:

- Connection and disconnection events
- Task creation (`created`)
- Data sent back to the client

The Makefile was updated to compile the `demo.c` file into an executable so it can be used for testing the scheduler. All components compile together using a single `make` command, ensuring consistency across the system.

3 Execution Instructions

3.1 Compilation

Navigate to the project directory and run:

```
make clean
make
```

This produces four executables:

- **myshell**: the local shell (Phase 1 functionality)
- **server**: the multithreaded scheduling server (Phase 4)
- **client**: the remote shell client
- **demo**: the demonstration program used to simulate long-running processes

3.2 Running the Server

Start the server on the remote machine:

```
./server
```

The server listens on port 8080 by default and prints:

```
[INFO] Server started, waiting for client connections...
[SCHEDULER] Initialised (RR q1=3s q*=7s + SJRF + FCFS)
[SCHEDULER] Thread started.
```

The server runs indefinitely, accepting unlimited simultaneous clients and scheduling their program tasks concurrently. Press `Ctrl+C` to stop.

3.3 Running the Client

In one or more separate terminals while the server is running:

```
./client
```

The client connects to `127.0.0.1:8080`, displays a `$` prompt, and accepts commands exactly as in previous phases. Type `exit` to disconnect.

3.4 Running the Demo Program Directly

The `demo` executable can also be run standalone for testing:

```
./demo N
```

where `N` is a positive integer. The program prints one line per second for `N` seconds in the format `Demo i/N`, then exits. When submitted to the server via the client, the scheduler treats `N` as the predicted burst time and schedules the process accordingly.

4 Testing

Below we start one server and three clients and, using `demo.c` and shell commands, test our server.

4.1 Test Scenarios

We complete the three test scenarios in the project description as follows:

4.1.1 Test Scenario 1: Three Clients Running Shell Commands

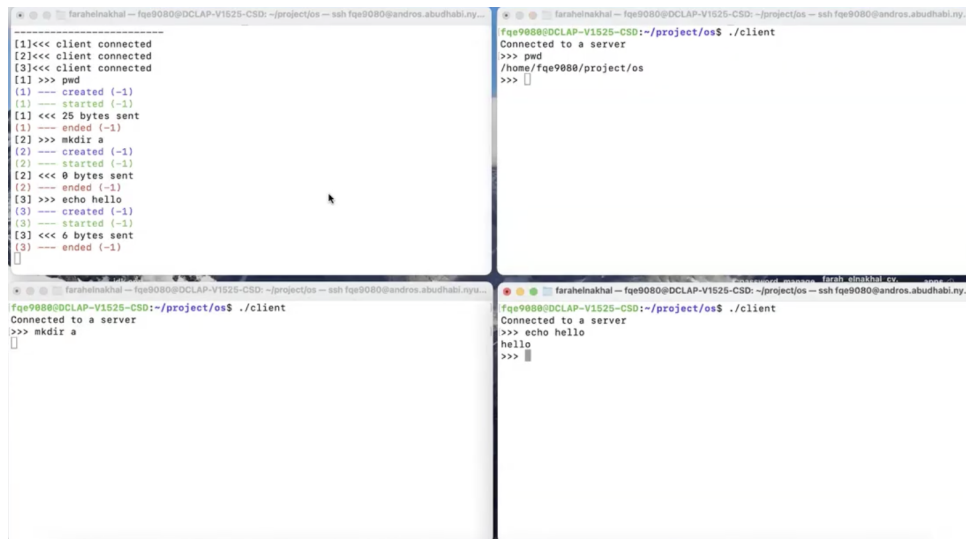
Open three separate terminals, start the server, then in each terminal connect a client and run a single shell command:

```
# Terminal 2
./client
$ pwd

# Terminal 3
./client
$ mkdir a

# Terminal 4
./client
$ echo hello
```

Since all three requests are shell commands (burst time = -1), each is executed immediately without preemption. The server log will show [N]--- created (-1), [N]--- started (-1), and [N]--- ended (-1) for each client in the order they are received. When running this test, we see the output outlined in Fig. 1. As all are shell commands, they executed immediately, bypassing scheduling entirely.



```
farahelnakhai -- fqe9080@DCLAP-V1525-CSD: ~/project/os -- ssh fqe9080@andros.abudhabi.ny...
[1]<<< client connected
[2]<<< client connected
[3]<<< client connected
[1]>>> pwd
[1] --- created (-1)
[1] --- started (-1)
[1]<<< 25 bytes sent
[1] --- ended (-1)
[2]>>> mkdir a
[2] --- created (-1)
[2] --- started (-1)
[2]<<< 0 bytes sent
[2] --- ended (-1)
[3]>>> echo hello
[3] --- created (-1)
[3] --- started (-1)
[3]<<< 6 bytes sent
[3] --- ended (-1)

farahelnakhai -- fqe9080@DCLAP-V1525-CSD: ~/project/os$ ./client
Connected to a server
>>> pwd
/home/fqe9080/project/os
>>>

farahelnakhai -- fqe9080@DCLAP-V1525-CSD: ~/project/os$ ./client
Connected to a server
>>> mkdir a

farahelnakhai -- fqe9080@DCLAP-V1525-CSD: ~/project/os$ ./client
Connected to a server
>>> echo hello
hello
>>>
```

Figure 1: Screenshot of output for Test Scenario 1.

4.1.2 Test Scenario 2: One Shell Command and Two Demo Programs

```
# Terminal 2
./client
$ pwd

# Terminal 3
./client
$ ./demo 12

# Terminal 4
./client
$ ./demo 12
```

The shell command executes immediately. The two demo processes are enqueued and scheduled in alternating rounds. The server log will show `waiting`, `running`, and `pre-emption` events for each demo process, along with the execution summary line at the end.

Below, in Fig. 2 we can see the output of running such a test. The shell command executes immediately and the two demo processes are enqueued.

```
[3] <<< 6 bytes sent
[3] --- ended (-1)
[1] >>> pwd
[1] --- created (-1)
[1] --- started (-1)
[1] <<< 24 bytes sent
[1] --- ended (-1)
[2] >>> ./demo 12
[2] --- created (12)
[2] --- waiting (12)
[2] --- running (12)
[3] >>> ./demo 12
[3] --- created (12)
[3] --- waiting (12)
[2] --- waiting (9)
[3] --- running (12)
[3] --- waiting (9)
[2] --- running (9)
[2] --- waiting (2)
[3] --- running (9)
[3] --- waiting (2)
[2] --- running (2)
[2] --- ended (0)
[2] <<< 133 bytes sent
[3] --- running (2)
[3] --- ended (0)
[3] <<< 133 bytes sent
Summary: 0)---2)---(12)---P3---(24)---

CLAP-V1525-CSD:~/os_project$ ./client
4:32
r a
cannot create directory 'a': File exists
no 12
2
Demo 1/12
Demo 2/12
Demo 3/12
Demo 4/12
Demo 5/12
Demo 6/12
Demo 7/12
Demo 8/12
Demo 9/12
Demo 10/12
Demo 11/12
Demo 12/12
>>>

mr7316@DCLAP-V1525-CSD:~/os_project$ ./client
Connected to a server
>>> pwd
/home/mr7316/os_project
>>>

mr7316@DCLAP-V1525-CSD:~/os_project$ ./client
Connected to a server
>>> echo hello
hello
>>> ./demo 12
Demo 9/12
Demo 1/12
Demo 2/12
Demo 3/12
Demo 4/12
Demo 5/12
Demo 6/12
Demo 7/12
Demo 8/12
Demo 9/12
Demo 10/12
Demo 11/12
Demo 12/12
>>>
```

Figure 2: Screenshot of output for Test Scenario 2.

4.1.3 Test Scenario 3: Three Demo Programs with Different N Values

```
# Terminal 2
./client
$ ./demo 12

# Terminal 3
./client
$ ./demo 10
```

```
# Terminal 4
./client
$ ./demo 8
```

All three processes are program tasks. The scheduler applies SJRF so the process with the shortest remaining time is always selected first. The process with $N = 8$ will finish earliest, followed by $N = 10$, then $N = 12$. The server log shows waiting, running, and preemption events for each client throughout execution, and the execution summary line is printed once all three finish.

Below, in Fig. 3 we can see the output of running this test case, where the scheduler prioritizes shorter remaining jobs first.

```
Summary: 0)->P2-(12)-P3-(24)-
[1] >>> ./demo 8
[1] --- created (8)
[1] --- waiting (8)
[1] --- running (8)
[2] >>> ./demo 10
[2] --- created (10)
[2] --- waiting (10)
[3] >>> ./demo 12
[3] --- created (12)
[3] --- waiting (12)
[1] --- waiting (5)
[2] --- waiting (7)
[1] --- running (5)
[1] --- ended (8)
[1]<<< 81 bytes sent
[2] --- running (7)
[2] --- ended (8)
[2]<<< 111 bytes sent
[3] --- running (12)
[3] --- waiting (9)
[3] --- running (9)
[3] --- waiting (2)
[3] --- running (2)
[3] --- ended (10)
[3]<<< 133 bytes sent
Summary: 0)->P1-(8)-P2-(10)-P3-(30)-

I to a server
r a
5:50 annot create directory 'a': File exists
no 12
2
Demo 4/12
Demo 3/12
Demo 4/12
Demo 5/12
Demo 6/12
Demo 7/12
Demo 8/12
Demo 9/12
Demo 10/12
Demo 11/12
Demo 12/12
>>> ./demo 10
Demo 8/10
Demo 3/10
Demo 2/10
Demo 3/10
Demo 4/10
Demo 5/10
Demo 6/10
Demo 7/10
Demo 8/10

Hello
>>> ./demo 12
Demo 8/12
Demo 1/12
Demo 2/12
Demo 3/12
Demo 4/12
Demo 5/12
Demo 6/12
Demo 7/12
Demo 8/12
Demo 9/12
Demo 10/12
Demo 11/12
Demo 12/12
>>> ./demo 12
Demo 8/12
Demo 1/12
Demo 2/12
Demo 3/12
Demo 4/12
Demo 5/12
Demo 6/12
Demo 7/12
Demo 8/12
Demo 9/12
Demo 10/12
Demo 11/12
Demo 12/12
```

Figure 3: Screenshot of output for Test Scenario 3.

4.2 Demo Video

A demonstration of test scenarios can be found here:

<https://www.loom.com/share/a12c28d52efa4d1c976ff07dc25937f0>

5 Challenges

One of the main challenges was correctly integrating the scheduler with the server without breaking the functionality from previous phases. Initially, there were issues where tasks would either not be scheduled correctly or would block the server when multiple clients were connected. This was resolved by ensuring that the scheduler runs in its own thread and that proper synchronization (mutex + semaphore) is used.

Another challenge was handling multiple clients running the demo program at the same time. Early versions would sometimes get stuck due to shared resources (such as pipe file

descriptors). This was fixed by making each task maintain its own pipe, avoiding conflicts between concurrent processes.

Ensuring correct logging and output formatting was also tricky. The assignment required very specific output formats (colors, state transitions, and summary structure), so several adjustments were made to match the expected format exactly.

Recording the demo video was also a challenge since NYU has Zoom and Meet recording options disabled for us.

6 Division of Tasks

Myra was responsible for the serverside integration of the system. This included implementing `server.c` and updating the `makefile`. She handled detecting incoming command types, forwarding `./demo N` tasks to the scheduler, and executing non-scheduler commands immediately. Additionally, she implemented serverside logging.

Farah was responsible for designing and implementing the scheduler engine. This included creating `scheduler.c`, `scheduler.h`, and `demo.c`. She developed the full scheduling algorithm combining Round Robin, Shortest Job Remaining First, and FCFS tie-breaking, along with preemption. She also implemented scheduler side logging, and generated the execution summary line.

Both partners collaborated on integrating the scheduler with the server, debugging system interactions, and performing final testing. They also worked together on compiling the final system, running test scenarios, capturing output screenshots, and completing the report.